Bellman equations

Reward now, value later

> **The value of a situation equals the expected reward from the next action, plus the discounted value of the situation that follows.**

This guide connects Bellman equations to our CartPole experiment and PPO's critic. Work through the numbers first; the notation describes the same idea. All numerical examples below are illustrative, not outputs from our saved model.

1. Reward, return, and value

CartPole gives +1 for each step, including the failure step in our default setup. But a nearly upright pole and a rapidly falling pole can have the same immediate reward and very different futures.

Reward	The number received from one transition.
Return	The accumulated rewards along an attempt, possibly discounted.
Value	The expected return from a starting situation under a specified policy.

The critic predicts value. It does not choose the action; that is the actor's job.

2. A three-step example

Suppose exactly three rewards remain before a true terminal state:

$$s_0 \xrightarrow{r=1} s_1 \xrightarrow{r=1} s_2 \xrightarrow{r=1} \text{terminal}.$$

With no discounting, the return is $1 + 1 + 1 = 3$. With discount $\gamma = 0.9$:

$$G_0 = 1 + 0.9(1) + 0.9^2(1) = 2.71.$$

From the next state, two rewards remain, so $G_1 = 1 + 0.9(1) = 1.9$. Therefore:

$$\boxed{G_0 = 1 + 0.9G_1 = 1 + 0.9(1.9) = 2.71.}$$

The remaining future has been summarized by one number. This decomposition is the starting point for Bellman's relationship.

3. Why it works

Let r_t be the reward received after the action at step t. Then

$$\begin{aligned} G_t &= r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \cdots \\ &= r_t + \gamma\big(r_{t+1} + \gamma r_{t+2} + \cdots\big) \\ &= \boxed{r_t + \gamma G_{t+1}}. \end{aligned}$$

Bellman equations apply this decomposition to *expected* returns.

4. The value belongs to a policy

A policy π is a rule for choosing actions. Starting from the same CartPole state, an always-right controller and a trained controller can have different expected returns.

$$V^\pi(s) = \text{expected discounted return from state } s, \text{ following } \pi.$$

The superscript π identifies the policy; it is not an exponent. Its Bellman equation is

$$\boxed{V^\pi(s) = \mathbb{E}[r_t + \gamma V^\pi(s_{t+1}) \mid s_t = s]\,.}$$

s	Current state.
s_{t+1}	Next state after the action and environment update.
r_t	Reward received from that transition.
γ	Discount factor; later rewards get smaller weights.
$\mathbb{E}$	Average over policy action choices and possible environment outcomes.

For this equation, the state must contain enough information to predict the next-state distribution from the current state and action. A partial sensor observation may need history or an estimator to provide that information. Do not assume Wall-E's pitch alone is a complete state.

5. Connect it to a stochastic policy

In a toy state, suppose the policy chooses left with probability 0.3 and right with probability 0.7. Each action leads to one definite next state in this example:

Action	Probability	Reward	Next value
Left	0.3	1	10
Right	0.7	1	20

With $\gamma = 0.9$, the reward-plus-next-value is $1 + 0.9(10) = 10$ for left and $1 + 0.9(20) = 19$ for right. Thus

$$\boxed{V^\pi(s) = 0.3(10) + 0.7(19) = 16.3.}$$

The average follows the policy's probabilities. One sampled action does not itself represent this entire expectation.

6. State value versus action value

$V^\pi(s)$ lets the policy choose the first action. $Q^\pi(s, a)$ fixes that first action to a, then follows π afterward:

$$Q^\pi(s, a) = \mathbb{E}[r_t + \gamma V^\pi(s_{t+1}) \mid s_t = s, a_t = a]\,.$$

For the toy state, $Q^\pi(s, \text{left}) = 10$ and $Q^\pi(s, \text{right}) = 19$. Their policy-weighted average gives $V^\pi(s) = 16.3$:

$$V^\pi(s) = \sum_a \pi(a \mid s) Q^\pi(s, a).$$

7. Evaluate a policy or find the best one?

The expectation equation asks how well a specified policy performs. The optimal value $V^*(s)$ asks for the best expected return achievable from that state:

$$\boxed{V^*(s) = \max_a \mathbb{E}[r_t + \gamma V^*(s_{t+1}) \mid s_t = s, a_t = a]\,.}$$

Policy evaluation: average action outcomes using the policy's probabilities.

Optimality: select the action with the greatest expected outcome, assuming optimal behavior afterward too.

An optimal action-value function obeys

$$Q^*(s,a) = \mathbb{E}\left[r_t + \gamma \max_{a'} Q^*(s_{t+1}, a') \mid s_t = s, a_t = a\right].$$

The maximum is over actions, not over lucky environment outcomes. The latter still require an expectation. PPO's critic approximates the current policy's value; it is not directly told the optimal value.

8. Learn from a one-step prediction error

Let $\hat{V}$ denote a critic's approximate value. Suppose a transition has

$$\hat{V}(s_t) = 15, \qquad r_t = 1, \qquad \hat{V}(s_{t+1}) = 20, \qquad \gamma = 0.9.$$

A one-step target is

$$y_t = r_t + \gamma \hat{V}(s_{t+1}) = 1 + 0.9(20) = 19.$$

The **temporal-difference error** compares that target with the old prediction:

$$\boxed{\delta_t = r_t + \gamma \hat{V}(s_{t+1}) - \hat{V}(s_t) = 4.}$$

A table-based update with learning rate $\alpha = 0.1$ would give

$$V_{\text{new}}(s_t) = 15 + 0.1(4) = 15.4.$$

A neural-network critic instead changes its weights to reduce prediction errors over many samples. Using an estimated next value in the target is **bootstrapping**. It avoids waiting for the whole future, but an inaccurate next prediction can make the target inaccurate too.

9. The end of an episode

After a true terminal transition, no future reward remains. Use $y_t = r_t$. The default CartPole failure step earns 1, so its one-step target is 1, even though the attempt failed.

Do not use the reset state's value as the continuation of the failed attempt. A time-limit truncation can be different: if it only interrupts an otherwise continuing task, the next value can still matter. If the deadline is part of the task itself, value depends on remaining time. This is why `terminated` and `truncated` need different handling.

10. Where this appears in our PPO experiment

PPO's Generalized Advantage Estimation (GAE) combines temporal-difference errors across steps. Away from episode boundaries, its pattern is

$$\hat{A}_t \approx \delta_t + (\gamma\lambda)\delta_{t+1} + (\gamma\lambda)^2\delta_{t+2} + \cdots .$$

The rollout length and episode boundaries determine where this sum ends. Our configuration uses $\gamma = 0.99$ and $\lambda = 0.95$.

Collect transitions and critic predictions
↓
Use rewards and next values to estimate advantages and return targets
↓
Train the critic toward the targets; train the actor using advantages
↓
Collect fresh experience with the updated policy

Positive estimated advantage favors increasing an action's probability; negative advantage favors decreasing it. PPO's clipping modifies the actor objective to reduce incentives for excessive policy changes. The critic uses rollout-based targets, not only the simple one-step target illustrated on page 3.

11. Why a score of 500 is not a value of 500

Our reported episode score sums undiscounted rewards. If exactly 500 rewards of +1 remain, the discounted sum at $\gamma = 0.99$ is

$$\sum_{k=0}^{499} 0.99^k = \frac{1 - 0.99^{500}}{1 - 0.99} \approx 99.34.$$

The episode score is still 500. The critic's output is an estimate, so it need not equal this exact hypothetical sum. Neither number establishes physical robot performance.

12. Keep these distinctions clear

- Reward is one transition; value summarizes the expected future.
- Bellman is a consistency relationship, not a neural-network architecture.
- A stochastic policy requires averaging across its possible choices.
- A positive TD error is a prediction surprise, not proof of causation.
- Bellman relationships do not require knowing the physics equations: samples can provide reward and next-state information.

References and project files

Primary reference: OpenAI Spinning Up, Part 1: Key Concepts in RL, sections on value functions, Bellman equations, and advantages. The official source text was checked for this guide.

Project connections:
`configs/ppo.yaml` — experiment settings.
`training/train.py` — learning and checkpoints.
`training/common.py` — environment and evaluation.
`docs/CARTPOLE_GYMNASIUM.md` — runnable walkthrough.

The actor runs at inference; the critic and optimizer support laptop training.